

Probar servicios con MockMvc

3. Probar servicios con MockMvc

Probar servicios con MockMvc

Hasta ahora has probado la API a mano: con curl (Actividad 1.1) y con Swagger UI (Actividad 1.2). Los dos funcionan, pero comparten un problema — tienes que repetir los mismos clics o comandos cada vez que quieres comprobar que todo sigue funcionando. Hoy conoces un tercer cliente, uno que se ejecuta solo: un test automatizado.

Qué es un test automatizado

Imagina que LibroService ya funciona bien, y hoy cambias algo en `update()` para arreglar un fallo. ¿Cómo sabes que ese cambio no ha roto `create()`? La forma manual es volver a probar cada endpoint a mano cada vez que cambias una línea.

Un test automatizado es código que hace esa comprobación por ti: lo escribes una vez, y lo repites en segundos cada vez que lo necesites, sin abrir el navegador ni recordar qué había que probar.

JUnit es la librería estándar para escribir y ejecutar tests en Java: escribes métodos que comprueban un comportamiento concreto, y JUnit los ejecuta todos y dice cuáles han pasado y cuáles no.

Preparar - actuar - afirmar (Arrange-Act-Assert)

```
@Test
void sumar_DebeDevolverLaSumaCorrecta() {
    Calculadora calc = new Calculadora();           // preparar

    int resultado = calc.sumar(2, 3);                // actuar

    assertEquals(5, resultado);                      // afirmar
}
```

`@Test` marca el método como un test que JUnit debe ejecutar. Primero preparas lo que necesitas, luego actúas (llamas al método que quieres probar), y por último afirmas con `assertEquals(esperado, actual)` — el primer parámetro es siempre el valor que tú esperas, el segundo el que de verdad ha devuelto tu código.

Por qué importa el orden de assertEquals

Si inviertes esperado y actual el test sigue funcionando igual, pero el mensaje de error, cuando falle, sale con los dos valores cambiados — confunde al leerlo. Cambia el 5 de arriba por un 6 (una afirmación incorrecta) y ejecuta el test otra vez:

```
org.opentest4j.AssertionFailedError:  
Expected :6  
Actual   :5
```

Esa es la razón de ser de un test: no solo avisa de que algo va mal, dice exactamente qué esperabas y qué ha pasado en realidad, sin imprimir valores por consola para averiguarlo tú mismo.

Qué es un mock

Piensa en una entrevista de trabajo simulada, antes de la real: alguien hace de entrevistador, siguiendo un guion decidido de antemano ("cuando te pregunte esto, responde aquello"), y tú practicas cómo reaccionar, sin que haya un puesto real en juego todavía.

Un mock es exactamente eso, para código: un objeto falso que sustituye a una dependencia real, programado por ti para responder exactamente lo que decidas cuando lo llamen de una forma concreta.

Un mock con Mockito

```
LibroService mockService = mock(LibroService.class);  
when(mockService.findAll()).thenReturn(List.of());
```

`mock(LibroService.class)` crea el objeto falso — por defecto, no sabe hacer nada por sí solo. `when(...).thenReturn(...)` es el "guion": le dices qué debe devolver cuando lo llamen de una forma concreta.

Dentro de un test de Spring, en vez de crear el mock a mano con `mock(...)`, se usa la anotación `@MockitoBean` sobre el campo — Spring lo crea e inyecta él solo, pero por debajo es el mismo mecanismo.

Por qué usar un objeto falso en vez del real

Porque aísla lo que quieres probar. Si tu test dependiera del LibroService real, dependería de una base de datos real conectada, con datos reales dentro — y el resultado del test cambiaría según qué datos hubiera en ese momento, algo que no controlas ni te interesa cuando lo único que quieres saber es "¿mi controller responde bien a lo que le da el service?".

Con un mock, el service siempre se comporta exactamente como tú has decidido, esté la base de datos levantada o no, sea la hora que sea.

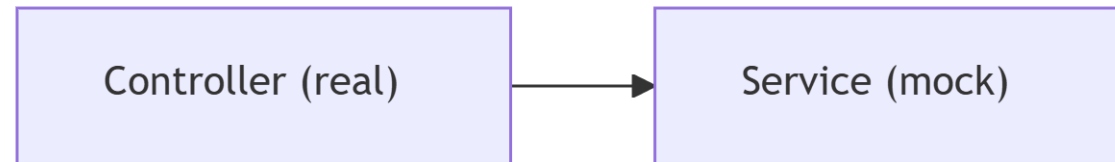
Test aislado vs. test de integración

Según las piezas que quieras incluir, puedes plantear el test de dos formas:

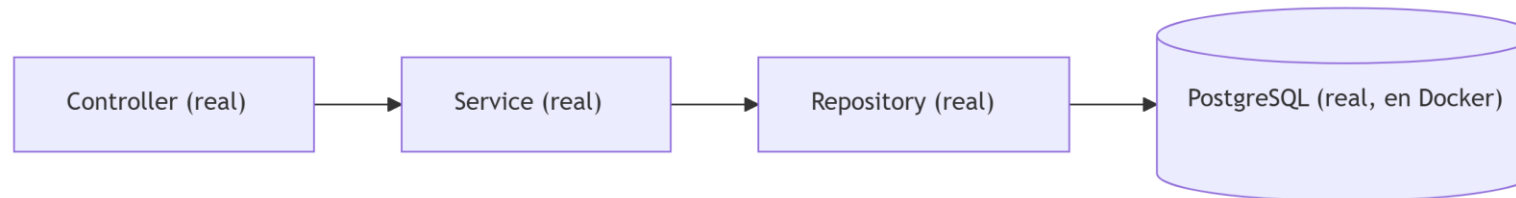
	Test aislado	Test de integración
Qué prueba	Una parte concreta de la aplicación	Varias partes reales trabajando juntas
Dependencias	Se sustituyen mediante mocks	Se utilizan las dependencias reales
Base de datos	No es necesaria	Puede utilizarse una base de datos de prueba
Velocidad	Muy rápido	Más lento

Hasta dónde llega el test dentro de la aplicación

Test aislado — @WebMvcTest



Test de integración — @Testcontainers



Qué cambia entre los dos

En el test aislado, el controller es real, pero el service se sustituye por un mock. De esta manera puedes comprobar cómo responde el controller sin depender de la lógica de negocio ni de una base de datos.

En el test de integración, la petición recorre varias piezas reales de la aplicación. Esto permite comprobar que funcionan correctamente en conjunto, aunque el test tarda más en ejecutarse.

Los tests unitarios que ya conoces son un tipo de test aislado. En este apartado aplicarás esa misma idea de aislamiento a un controller mediante `@WebMvcTest`.

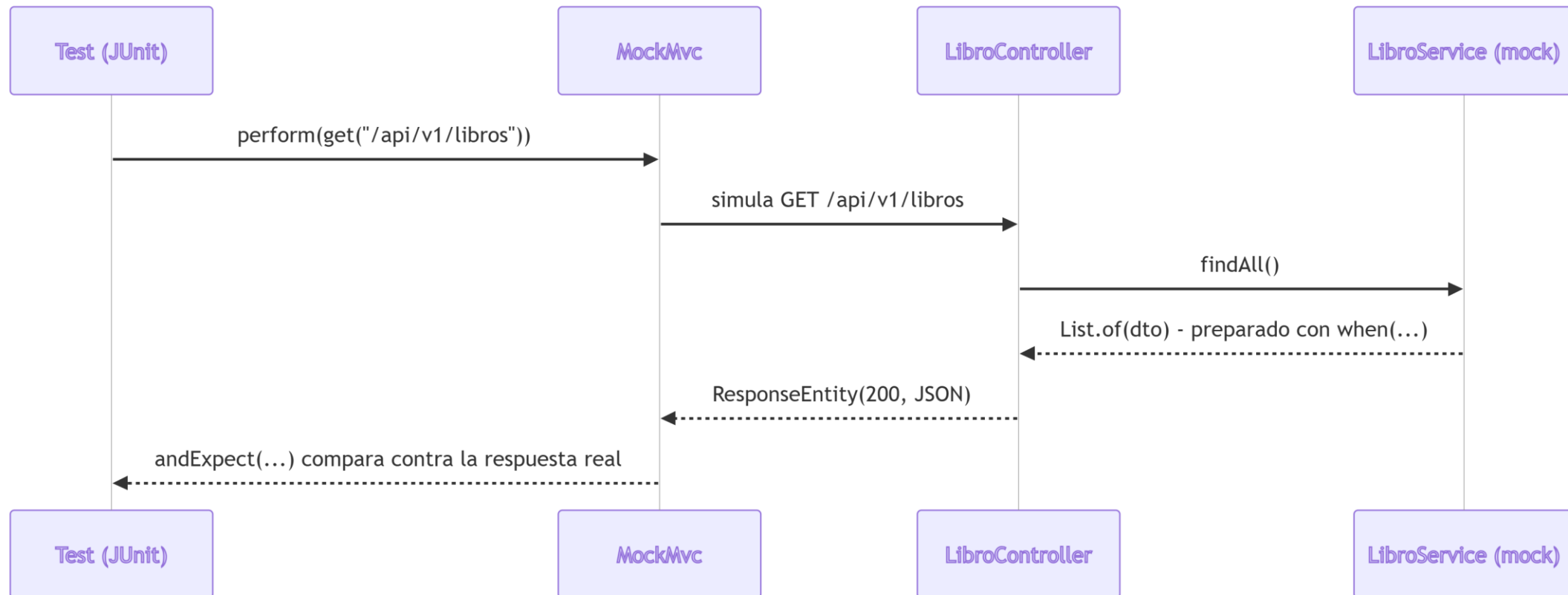
MockMvc: un cliente HTTP programable

MockMvc es la herramienta de Spring para probar controllers REST sin necesitar un servidor HTTP real arrancado: simula peticiones HTTP contra tus controladores, dentro del propio test, y permite comprobar el código de estado y el cuerpo de la respuesta con código Java.

Es, en esencia, un tercer cliente — como curl o Swagger UI — pero con una diferencia clave: es repetible y se puede ejecutar automáticamente (en tu máquina o en un pipeline de CI) cada vez que cambias algo, sin que nadie abra el navegador.

El viaje completo de una petición MockMvc

Fíjate en que nunca sale del propio proceso Java, no hay ningún puerto 8080 real de por medio:



Qué simula cada pieza

MockMvc simula la parte de Tomcat (recibir la petición, encaminarla al método correcto) sin arrancar ningún servidor de verdad; LibroController es exactamente el mismo código que corre en producción, sin cambiar una línea; y LibroService, al estar mockeado, responde con el valor que tú has preparado en el test — no con nada calculado de verdad.

Primer ejemplo: un test que sí necesita el mock

```
@WebMvcTest(LibroController.class)
@AutoConfigureMockMvc(addFilters = false)
class LibroControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private LibroService libroService;

    @Test
    void getAll_DebeDevolverListaDeLibros() throws Exception {
        var dto = new LibroResponseDTO(1L, "El nombre del viento", ...);

        when(libroService.findAll()).thenReturn(List.of(dto));

        mockMvc.perform(get("/api/v1/libros"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].titulo").value("El nombre del viento"));
    }
}
```

Pieza a pieza (1/2): preparar el entorno

- `@WebMvcTest(LibroController.class)`: prepara un entorno reducido para probar el controller sin arrancar toda la aplicación ni conectarse a una base de datos real. Spring carga solo lo necesario para recibir la petición, dirigirla y construir la respuesta.
- `@AutoConfigureMockMvc(addFilters = false)`: desactiva los filtros de servlet que se aplican a cada petición — en concreto, los que añadirá Spring Security a partir del Tema 2. Sin este flag, en cuanto el proyecto tenga seguridad, estos tests fallarían con un 401 en vez del código que de verdad quieres comprobar.
- `@MockitoBean private LibroService libroService`: sustituye el service real por un mock, para decidir qué devolverá y comprobar únicamente cómo reacciona el controller.

Pieza a pieza (2/2): preparar - actuar - afirmar

- `when(libroService.findAll()).thenReturn(List.of(dto))`: la parte de preparar. Le dices al mock, explícitamente, qué debe devolver cuando alguien lo llame con `findAll()` — no hay ninguna base de datos decidiéndolo, decides tú, a mano, exactamente qué datos ve el controller.
- `mockMvc.perform(get(...))`: construye y envía una petición simulada — el equivalente, en código, al curl que ya conoces.
- `.andExpect(status().isOk()) / .andExpect(jsonPath(...))`: la parte de afirmar. `jsonPath` navega el cuerpo JSON de la respuesta como un mini-selector — aquí, `$.title` es el campo título del primer elemento del array que devuelve `findAll()`.

Segundo ejemplo: cuando el mock no hace falta

```
@Test
void create_DebeDevolver400_CuandoElDtoNoEsValido() throws Exception {
    String body = ""
        { "titulo": "", "precio": -5,
          "fechaPublicacion": null, "editorialId": -1 }
        "";

    mockMvc.perform(post("/api/v1/libros")
        .contentType(MediaType.APPLICATION_JSON)
        .content(body)
        .andExpect(status().isBadRequest()));
}
```

Fíjate en que este test no utiliza `when(...)`. La validación de `@Valid` se ejecuta antes de que el controller llame a `libroService.create(...)`: como el cuerpo incumple varias restricciones del DTO, Spring detiene la petición y responde 400 sin que el service llegue a intervenir.

Instruyendo al mock para que falle: thenThrow

thenReturn prepara un valor de éxito. Para el caso "no encontrado", preparas una excepción en su lugar:

```
@Test
void getById_DebeDevolver404_CuandoNoExiste() throws Exception {
    when(libroService.findById(999L))
        .thenThrow(new ResponseStatusException(
            HttpStatus.NOT_FOUND, "Libro no encontrado"));

    mockMvc.perform(get("/api/v1/libros/999"))
        .andExpect(status().isNotFound());
}
```

.thenThrow(...) en vez de .thenReturn(...): en vez de decirle al mock qué devolver, le dices qué excepción lanzar. Reproduce, sin tocar la base de datos, el mismo camino que ya conoces del `orElseThrow(...)` real de `LibroService`.

¿Y el test de integración completo?

Existe otro tipo de test, con `@Testcontainers`, que levanta bases de datos reales en contenedores Docker solo para la duración del test — no mockea nada. Lo trabajarás a fondo en Acceso a Datos; aquí basta con que sitúes los dos niveles: `@WebMvcTest` prueba una capa aislada y rápida, un test de integración prueba varias piezas reales trabajando juntas y es más lento pero da más confianza sobre el sistema completo.

Tu propio proyecto ya tiene el CRUD completo de Videojuego y de Estudio, documentado además con `@ApiResponse`s (Actividad 1.2) — en la Actividad 1.3 no construyes ningún endpoint nuevo, escribes los tests `MockMvc` que comprueban con código que esos mismos códigos que has documentado (200, 201, 400, 404...) son, de verdad, los que tu API devuelve. La Actividad 1.4 cierra el tema con dos piezas más: cuánto tarda tu API en atender varias peticiones a la vez, y `Actuator`.

- Un test automatizado comprueba comportamiento sin intervención humana, y se puede repetir en cada cambio; sigue el patrón preparar-actuar-afirmar. `assertEquals(esperado, actual)`: el orden importa para que el mensaje de error, si falla, se lea bien.
- Un mock es un objeto falso que sustituye a una dependencia real, programado por ti (`when(...).thenReturn(...)`) para aislar lo que quieres probar de todo lo demás.
- Un test aislado se centra en una parte concreta de la aplicación y sustituye sus dependencias mediante mocks. Un test de integración prueba varias piezas reales funcionando juntas.

Ideas clave (cont.)

- MockMvc simula peticiones HTTP contra tus controladores sin arrancar un servidor real — un cliente HTTP programable y repetible.
- `@WebMvcTest` arranca solo la capa web; `@AutoConfigureMockMvc(addFilters = false)` desactiva los filtros de seguridad; `@MockitoBean` sustituye una dependencia por un mock; `mockMvc.perform(...).andExpect(...)` construye la petición y afirma el resultado; `jsonPath` navega el cuerpo JSON.
- `when(mock.metodo(...)).thenReturn(valor)` prepara un valor de éxito; `.thenThrow(excepcion)` prepara que el mock lance una excepción (el mismo camino que `orElseThrow(...)` en el service real). No todos los tests necesitan `when(...)`: uno que verifica `@Valid` puede fallar antes de llegar al service.